

Práctica 1 — Ejercicio 2: Temporización por Hardware (Timer0)

Decisiones de Diseño y Arquitectura de Firmware

1. Introducción y Requerimientos

El objetivo consiste en reestructurar el control de entradas y salidas digitales implementado mediante bucles bloqueantes (`_delay_ms`), migrando hacia una arquitectura gobernada por interrupciones de hardware utilizando el módulo **Timer0** del ATmega328P.

- **Estado inicial:** Ambos LEDs en `PB0` y `PB1` totalmente encendidos.
- **Entrada:** Pulsadores en `PD2` y `PD3` con pull-ups internas habilitadas.
- **Comportamiento:** Al detectar pulsación, parpadeo alternado estricto de 250 ms por estado.
- **Condición técnica:** Temporización controlada por hardware sin demoras por software.

2. Decisiones de Diseño

A. Modo CTC frente a Modo Normal

Se seleccionó el modo **CTC (Clear Timer on Compare Match)** con el comparador `OCR0A` en lugar del desbordamiento normal (Overflow):

- El modo normal cuenta libremente hasta 255 (256 ciclos). A 16 MHz no genera múltiplos enteros exactos de 1 ms, acumulando error temporal con cada interrupción.
- El modo CTC reinicia el contador a cero por hardware de forma exacta al igualar el valor de `OCR0A` , sin requerir recargas por software dentro de la ISR.

B. Base de Tiempo Exacta de 1 ms

Para lograr 250 ms precisos, se configuró una interrupción patrón de 1 ms:

```
f_timer = 16 MHz / 64 = 250 kHz → Período (tick) = 1 / 250 kHz = 4 µs
Cuentas requeridas = 1000 µs / 4 µs = 250 ciclos
OCR0A = 250 - 1 = 249
```

Dentro de la ISR (`TIMER0_COMPA_vect`), un acumulador cuenta 250 interrupciones de 1 ms para completar los **250 ms exactos**.

C. Calificador volatile

Las variables globales compartidas entre el flujo principal y la interrupción (`modo` , `ticks_ms` , `toggle`) se declararon como `volatile` para impedir que el compilador optimice sus lecturas en registros internos de la CPU, asegurando acceso sincrónico a la SRAM.

3. Comparativa Arquitectónica

Parámetro	Ejercicio 1 (Polling + Delays)	Ejercicio 2 (Timer0 + ISR)
Uso de CPU	Bloqueada continuamente en el bucle <code>_delay_ms(250)</code> .	100% libre en el <code>while(1)</code> para atender eventos o modo sleep.
Latencia de pulsadores	Alta (hasta 250 ms); puede omitir pulsaciones cortas.	Inmediata ($< 1 \mu\text{s}$); respuesta instantánea al presionar.
Precisión temporal	Degradada por tiempo de ejecución de instrucciones adicionales.	Precisión fija gobernada por el cristal de 16 MHz.

4. Verificación en Proteus 8

- El campo **Clock Frequency** del microcontrolador debe configurarse en 16MHz para que la escala de tiempos coincida con `F_CPU` .
- La conmutación de los pines `PB0` y `PB1` debe contrastarse contra la barra inferior **Simulation Time**, confirmando que cada cambio de estado toma exactamente 0.250 s de tiempo simulado.